

Heaven's Light is Our Guide
Rajshahi University of Engineering and Technology



Course Code
ECE 3112

Course Title
Microprocessor, Assembly Language & Interfacing

Assignment Report:
8086-Based System for Controlling DC Motor Speed Using PWM Signal

Submitted to
Md. Omaer Faruq Goni
Assistant Professor
Dept of ECE, Ruet

Submitted by
Md. Tajim An Noor
Roll: 2010025

Abstract

This project demonstrates the design and implementation of an 8086 based system for generating a PWM signal to control the speed of a DC motor. The system utilizes various peripheral devices, including 8255 PPI, 8253 PIC, ADC0804, and L298 motor driver, to achieve precise control. A potentiometer provides the user input for speed variation, which is translated into a digital signal using ADC0804. This digital value is used to adjust the PWM duty cycle dynamically, achieving real-time speed control. The report details the methodology, the hardware components used, and the process for integrating these elements to develop an efficient solution.

Contents

1	Introduction	1
2	Objective	2
3	Flow Chart	2
4	Implementation	2
4.1	Methodology	2
4.1.1	Initialization:	2
4.1.2	ADC Conversion:	3
4.1.3	PWM Signal Generation:	3
4.1.4	PWM Signal Generation Logic	3
4.1.5	Motor Control:	4
4.1.6	User Input Handling:	4
5	Code	5
6	Circuit Schematic	7
7	Simulation Outputs	8
8	Outcomes	10
9	Conclusion	10

1 Introduction

In modern control systems, the ability to control the speed of a DC motor is a fundamental requirement for various industrial and domestic applications. This project aims to develop an 8086 microprocessor-based system capable of generating a Pulse Width Modulation (PWM) signal to control a DC motor's speed.

The system incorporates the following key components:

- **8086 Microprocessor:** Serves as the core processing unit for executing the control logic.
- **Latches:** Three latches are used for address latching.
- **Transceivers:** Two transceivers are used for data latching.
- **Decoders:** One for read/write control and another for address decoding to select specific chips in the system.
- **8253 PIC:** Generates the PWM signal by using Counter 0 in rate generator mode and Counter 1 in monostable multivibrator mode.
- **8255 PPI:** Interfaces with I/O devices, allowing communication between the processor and peripherals.
- **Potentiometer Circuit:** Provides a variable resistance, converted to voltage, as an analog input for speed control.
- **ADC0804:** Converts the analog voltage from the potentiometer to a digital value for use in the microprocessor code.
- **L298 Motor Driver:** Drives the DC motors based on the PWM signal.
- **DC Motors:** The load whose speed is controlled.
- **Push Button:** Provides user input for speed adjustment.
- **Proteus 8.17:** Proteus Design Suite, version 8.17, was used as the simulation software.

This report details the methodology, system design, and implementation, alongside the challenges encountered and lessons learned.

2 Objective

The objective of this task is to design and implement an 8086 microprocessor-based system to control the speed of a DC motor using Pulse Width Modulation (PWM). The system incorporates various hardware components, including the 8255 Programmable Peripheral Interface (PPI), 8253 Programmable Interval Timer (PIC), ADC0804, and L298 motor driver, to achieve precise speed control. The goal is to provide a real-time, adjustable control mechanism for the motor speed by varying the duty cycle of the PWM signal, which is determined by the input from a potentiometer.

3 Flow Chart

The following is the flow chart of the process.

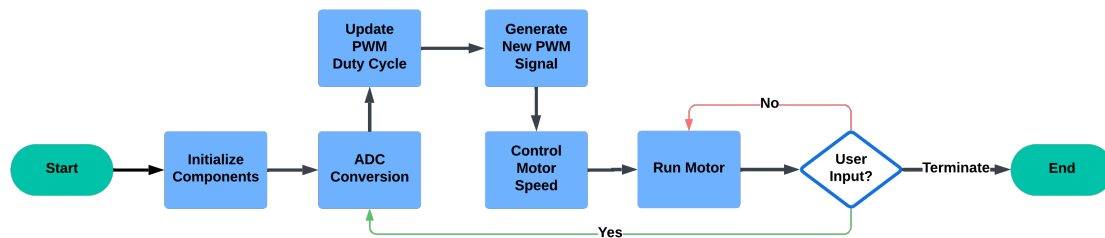


Figure 1: Process Flowchart

4 Implementation

4.1 Methodology

The following steps outline the system's methodology:

4.1.1 Initialization:

- Ports of the 8255 PPI are defined as follows: Port A as output, Port B as input, and Port C upper as input, Port C lower as output.
- Counter 0 of the 8253 PIC is initialized as a rate generator with a count of 0101H which is 0FFH+1H.
- Counter 1 of the 8253 PIC is initialized in mode 1 (monostable multivibrator). No count is given initially.

4.1.2 ADC Conversion:

In ADC0804, there are three pins, $\overline{CS}$, $\overline{WR}$, $\overline{RD}$ $\overline{INTR}$. Using these pins, the conversion can be started, and the digital value can be read by the processor. When $\overline{WR}$ and $\overline{CS}$ is Both low, the ADC is reset. When $\overline{WR}$ becomes high, the conversion starts and $\overline{INTR}$ goes high until conversion finishes. $\overline{INTR}$ goes low when conversion is done. To read the digital value, the $\overline{CS}$ and $\overline{RD}$ both need to be low. This logic is used to control the ADC.[1]

- The ADC is reset using the $\overline{CS}$ and $\overline{WR}$ pins via Port A of the PPI.
- The ADC is started for analog-to-digital conversion using the potentiometer's voltage as input.
- The system waits for the End-of-Conversion (EOC) signal from the ADC (via the $\overline{INTR}$ pin).
- Once conversion is complete, the digital value from the ADC is used to set the count of Counter 1 in the PIC.

4.1.3 PWM Signal Generation:

- Counter 0 in mode 2 (rate generator) generates a negative pulse when the count reaches 1.[2]
- This pulse is inverted and used as the trigger for Counter 1's gate signal.
- Counter 1, in monostable multivibrator mode, generates a PWM signal with a duty cycle proportional to the digital value from the ADC.

4.1.4 PWM Signal Generation Logic

The process of generating the PWM signal is as follows:

- **Counter 0 in Mode 2 (Rate Generator):** Counter 0 is configured in mode 2, where it generates one negative pulse each time it reaches a count of 1. For example, if the count is set to 101, Counter 0 will generate a negative pulse when the count reaches 100.[2]
- **Inverting Counter 0 Output:** The output from Counter 0 is inverted and then fed as the gate signal to Counter 1.
- **Counter 1 in Mode 1 (Monostable Multivibrator):** Counter 1 is initialized in mode 1. The count value for Counter 1 is dynamically taken from the ADC, which represents the desired PWM duty cycle. For instance:
 - If the ADC value is 50, Counter 1 will count 50 while staying low.

- The rest of the time, Counter 1 will remain high.
- **Triggering Counter 1:** The trigger for Counter 1's gate is the inverted output from Counter 0. For example:
 - Counter 0, with a count of 101, triggers a pulse at count 100.
 - This pulse starts the count in Counter 1.
- **Duty Cycle Calculation:** Counter 1 remains high for the full duration of the count (e.g., 100 in this example). When triggered, it goes low and starts counting the value set by the ADC (e.g., 50). As a result:
 - The low time of the PWM signal is $(100 - 50) = 50$.
 - The high time is the remaining time, giving a 50% duty cycle in this case.
- **PWM Output:** This PWM signal is then fed to the L298 motor driver to control the speed of the connected DC motor. The duty cycle dynamically changes as the ADC input varies, allowing real-time speed adjustment.

4.1.5 Motor Control:

- The generated PWM signal is fed to the L298 motor driver to control the speed of the connected DC motor.
- The DC motor used require 12V as input, which is given as a input in the motor driver.

4.1.6 User Input Handling:

- The system continuously checks for input from the push button via Port C of the PPI.
- On valid input, the ADC conversion process is repeated to adjust the PWM signal dynamically.
- This feature is implemented as the $\overline{INTR}$ stays high during conversion. If the ADC is always running and the microprocessor continuously reads the digital value, it may give unexpected value which in turn won't control the speed as intended. Its seen in simulation that without this stop-to-change feature, the speed control does not work as intended.

5 Code

```
1  DATA SEGMENT                                ;Defining port addresses
2      PPI_A EQU 80H
3      PPI_B EQU 82H
4      PPI_C EQU 84H
5      PPI_CTRL EQU 86H
6      PIC_CNT0 EQU 60H
7      PIC_CNT1 EQU 62H
8      PIC_CNT2 EQU 64H
9      PIC_CTRL EQU 66H
10 DATA ENDS
11
12 CODE SEGMENT
13     ASSUME CS:CODE, DS: DATA
14     MOV AX, DATA
15     MOV DS, AX
16 START:
17     MOV AL, 10001011B ;PPI control: PortA-Out; PortB-In;
18     ↪ PortC_Upper-In; PortC_Lower-Out
19     OUT PPI_CTRL, AL
20     MOV AL, 00110100B ;Counter 0, in mode 2, Rate_Gen;
21     ↪ LSB+MSB
22     OUT PIC_CTRL, AL
23     MOV AL, 01h ;LSB 01
24     OUT PIC_CNT0, AL ;MSB 01; count = 0101 = 101, in hex
25     ↪ mode, (FF+1)
26     MOV AL, 01h
27     OUT PIC_CNT0, AL
28     MOV AL, 01010010B ;Counter 1, in mode 1, Monostable,
29     ↪ only LSB
30     OUT PIC_CTRL, AL
31 CHNG:
32     MOV AL, 00000010B ;Reset ADC
33     OUT PPI_A, AL
34     MOV AL, 00000111B ;Start conversion
35     OUT PPI_A, AL
36     INTR_CHK: ;Wait till EOC
```



```

37             IN AL, PPI_C           ;Check INTR of ADC to see if EOC
38             AND AL, 01H
39             CMP AL, 00H
40             JNE INTR_CHK
41
42             MOV AL, 00000100B       ;Read value from ADC
43             OUT PPI_A, AL
44             IN AL, PPI_B            ;Change counter 1's count
45             OUT PIC_CNT1, AL
46
47 BACK:
48             IN AL, PPI_C           ;Input ADC Value, percentage of
49             ↪ PWM
49             CALL CHECK              ;If push button pressed,
49             ↪ change speed value
50             JMP BACK               ;Else continue
51
52 CHECK PROC                                ;Change counter 1's count
53             IN AL, PPI_C
54             AND AL, 80H
55             CMP AL, 00H
56             JZ CHNG
57             JNZ BACK
58 CHECK ENDP
59 CODE ENDS
60 END START

```

6 Circuit Schematic

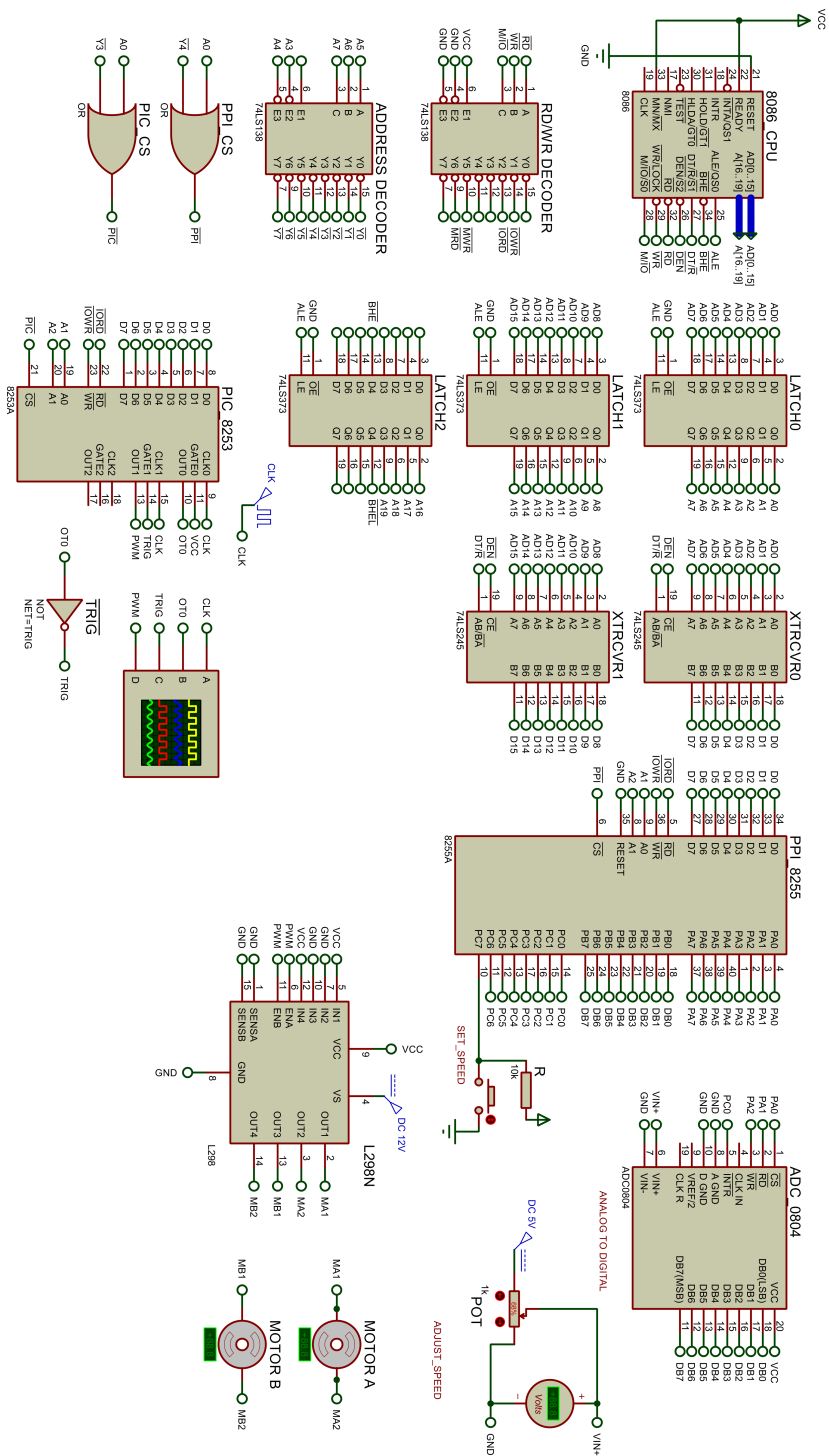


Figure 2: Circuit Schematic of the System

7 Simulation Outputs

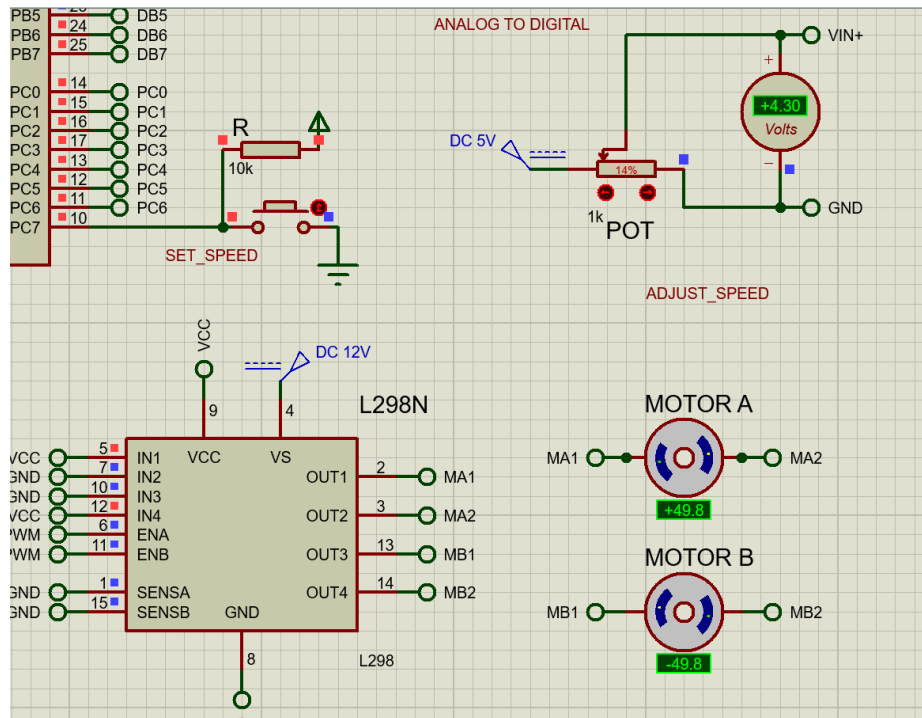


Figure 3: Speed at 14% PWM

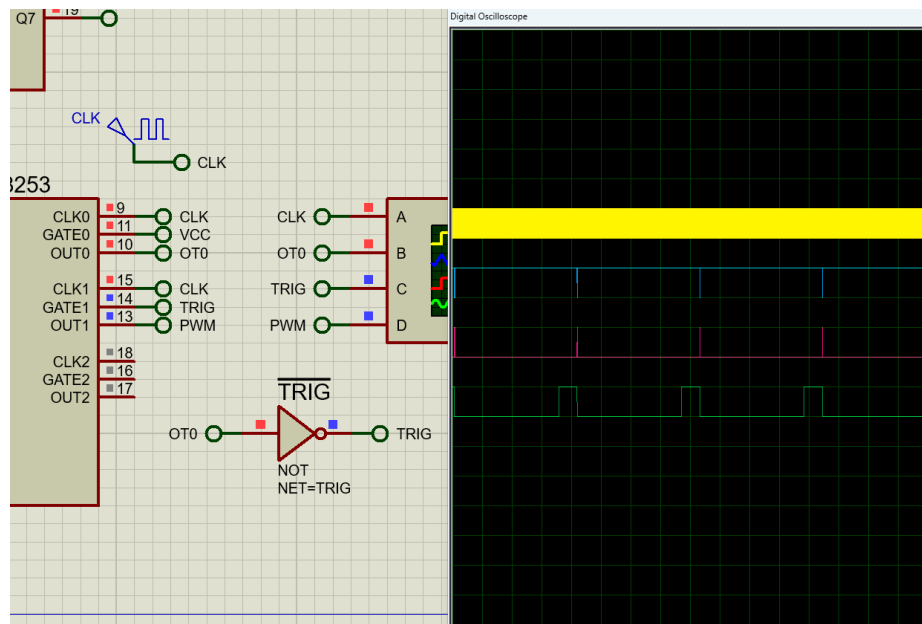


Figure 4: 14% PWM signal output

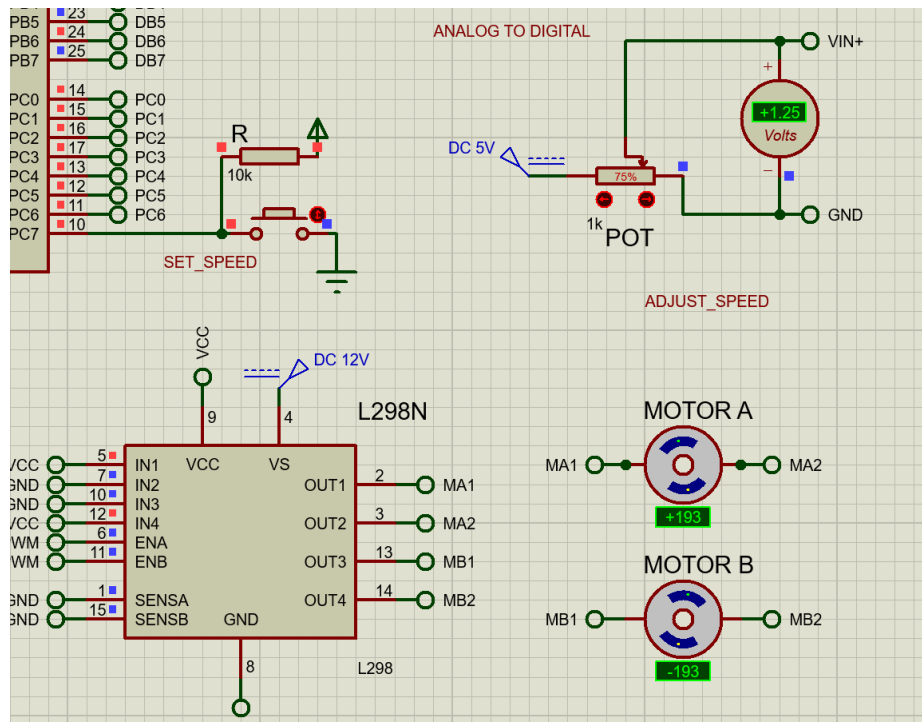


Figure 5: Speed at 75% PWM

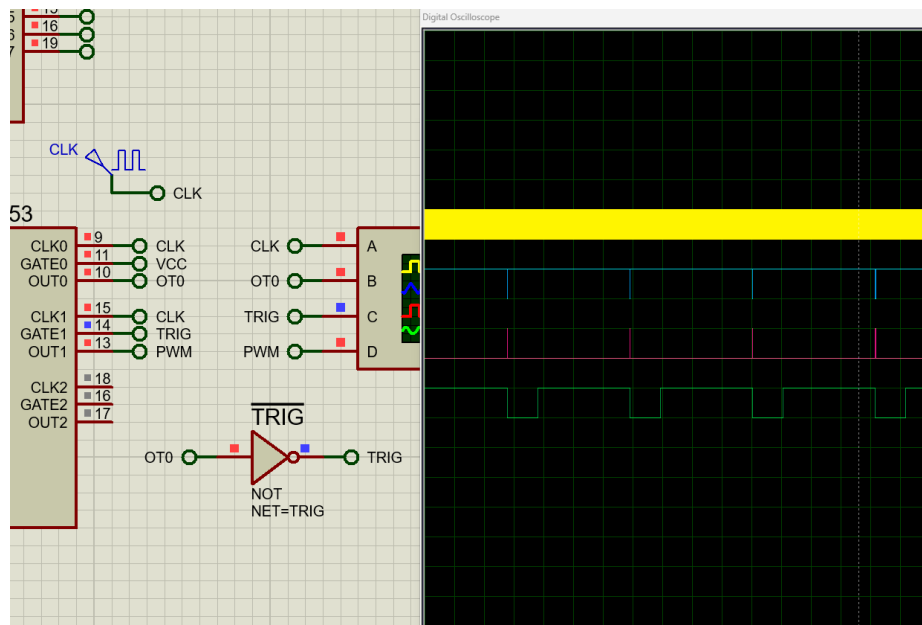


Figure 6: 75% PWM signal output

8 Outcomes

- Successfully implemented an 8086-based system for controlling the speed of a DC motor using PWM.
- Demonstrated the integration of various components, such as the 8255 PPI, 8253 PIC, ADC0804, and L298 motor driver, to achieve a functional control system.
- Learned the process of configuring counters in different modes (rate generator and monostable multivibrator) to generate a PWM signal with a variable duty cycle.
- Gained insights into the conversion of analog signals to digital using ADC0804 and the role of the potentiometer in varying the motor speed.
- Developed an understanding of hardware interfacing and real-time control logic using assembly language.
- Enhanced troubleshooting skills during simulation in Proteus and debugging the assembly code.

9 Conclusion

The design and implementation of an 8086-based system for DC motor speed control were successfully achieved. The use of PWM allowed precise control of motor speed, with the duty cycle dynamically adjusted based on user input through a potentiometer and finalized by another user input that uses a push button. The integration of peripherals such as 8255 PPI, 8253 PIC, and ADC0804 facilitated seamless communication and control within the system.^[2]

This project provided valuable insights into hardware-software integration, the functioning of microprocessor-based systems, and the practical application of PWM for motor control. It also enhanced understanding of assembly language programming and peripheral interfacing.

Future improvements could involve using a more advanced microcontroller or integrating additional sensors for more sophisticated control.

References

- [1] *ADC0804: 8-Bit P-Compatible A/D Converters*, National Semiconductor, 2002, datasheet. [Online]. Available: <https://www.ti.com/lit/ds/symlink/adc0804.pdf>
- [2] D. V. Hall, *Microprocessor and Interfacing: Programming and Hardware*, 2nd ed. New York: McGraw-Hill Education, 1986.